

AXI NOC with embedded ECC and HARQ

Lalit Prasad Peri, Alberta Dadeboe, Priyatam Annambhotla

Group5 - Final Project Report. ECE5545 Spring2024

Abstract. This project presents a study focusing on enhancing the reliability of on-chip Networks-on-Chip (NOCs) through the implementation of scalable AXI NOC with embedded Error Correction Code (ECC) and Hybrid Automatic Repeat request (HARQ) mechanisms. By introducing ECC and HARQ logic into a 4x4 NOC configuration, we observe a mere 4% increase in FPGA Look-Up Table (LUT) utilization, indicating efficient resource utilization. Furthermore, the integration of multi-stage clock-gating exhibits minimal area overhead, merely 1.7%, while concurrently achieving a substantial 26% reduction in dynamic power consumption at an 800MHz frequency corner. These findings underscore the feasibility and effectiveness of our proposed enhancements in enhancing NOC reliability while maintaining resource efficiency and power optimization. Moreover, we propose extending this project to investigate the scalability potential, particularly exploring configurations up to 256x256, to further advance high-speed NOC architectures.

1. Architecture and Design of AXI NOC

Key design features of a 4x4 AXI NOC includes:

1. **Multiple Masters:** The interconnect allows multiple master devices, such as CPUs, DMA controllers, or accelerators, to initiate data transactions independently. Each master has his own dedicated clock inputs to dynamically scale frequencies based on workload requirements.
2. **Multiple Slaves:** Similarly, the interconnect supports communication with multiple slave devices, which could include memory controllers, peripheral interfaces, or other functional blocks within the SOC. Each slave agent is physically addressed with 4KB of address space.
3. **Switching Arbitration:** The interconnect employs arbitration mechanisms to manage simultaneous access requests from multiple masters. This ensures fair and efficient utilization of the available bandwidth, preventing bottlenecks and contention for shared resources.
4. **Network Interface and Buffering:** Network interface manages communication between IP cores and the NOC, handling data packetization, addressing, and routing, while buffering temporarily stores packets to manage transmission delays and alleviate contention, ensuring efficient data transmission in complex System-on-Chip (SoC) designs.

2. Architecture and Design of SECDED ECC and Embedded HARQ

The principle of SECDED is to allow for the correction of any single-bit errors that may occur within a memory word, while also detecting the presence of double-bit errors.

2.0.1. Single Error Correction

SECDED can detect and correct any single-bit errors that occur within the memory word. When data is read from memory,

the ECC logic compares the calculated redundant bits with the received data bits. If a single-bit error is detected, the ECC logic can determine which bit is in error and correct it by flipping the incorrect bit back to its correct state.

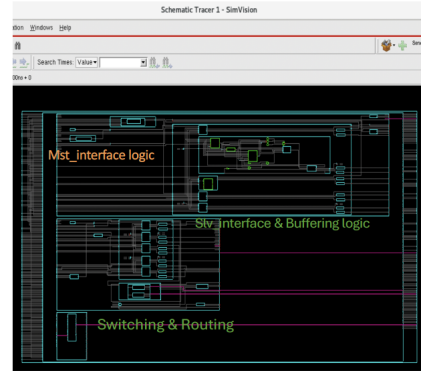


Figure 1. AX-NOC schematic Diagram

2.0.2. Double Error Detection

In addition to single error correction, SECDED is designed to detect the presence of double-bit errors within a memory word. If two or more bits within the memory word are corrupted, SECDED can detect that an error has occurred, but it cannot correct the error. Instead, the system typically triggers an error notification or takes corrective action, such as reloading the affected data from a redundant copy or requesting a re-transmission.

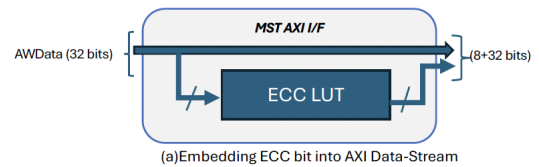


Figure 2. Master Interface-ECC Encode

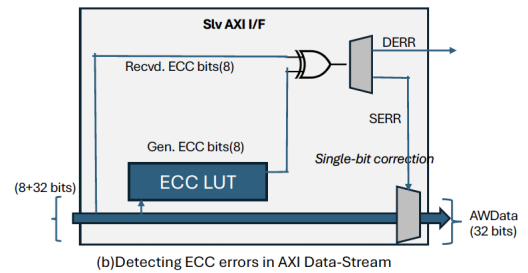


Figure 3. Slave Interface-ECC Decode

2.0.3. Embedded HARQ

Hybrid Automatic Repeat request, is a method utilized in communication systems to enhance the reliability and efficiency of

data transmission across unreliable channels like wireless networks. By integrating aspects of both forward error correction (FEC) and automatic repeat request (ARQ) mechanisms,

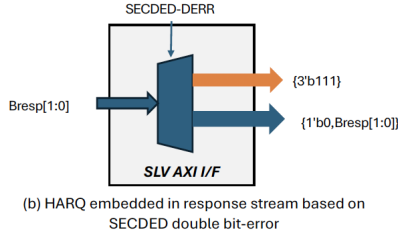


Figure 4. Embedded HARQ Response

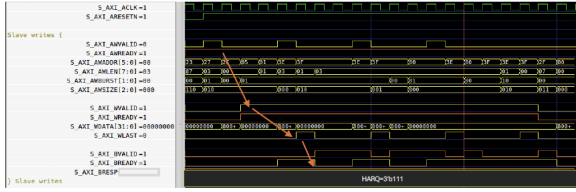


Figure 5. AXI-Write HARQ Response on DERR

HARQ response enhances the reliability and throughput of the Network-on-Chip (NOC) by selectively requesting retransmission of data only when multi-bit errors can be remedied by the self-correcting Single Error Correction, Double Error Detection (SECEDED) logic. Single-bit errors in the data stream are automatically corrected by the SECEDED logic, while double-bit errors are merely flagged. This flag can then prompt the alteration of slave response to signal a retransmission request. The HARQ response is encoded into the SLV RRESP/BRESP bus for efficient communication and coordination within the system.

3. Logic Synthesis and Static Timing Analysis

The main objectives of logic synthesis are to optimize the design for factors such as area, power, and timing, while ensuring that the functionality specified by the HDL description is preserved.

3.0.1. Steps in logic synthesis

During the design phase, the HDL description is parsed and analyzed to extract structural and behavioral information such as modules, signals, and connections. This is followed by optimization techniques that improve performance and efficiency, such as technology mapping for selecting appropriate gates and flip-flops, Boolean logic optimization to simplify equations, and timing optimization to fulfill limitations. The optimized design is then mapped onto the target technology library, resulting in a netlist that serves as a structural representation of the circuit.

Synthesis was performed for both the unoptimized and optimized designs for the baseline logic and the logic with ECC and HARQ mechanisms included.

The Xilinx Vivado (2023.2) Design tool was used with the AMD Virtex-7 xc7v2000tflg9125-2 FPGA product part. The

I/O was 1263 and the clock buffer count was 9 (not included figure 6 for easier visibility).

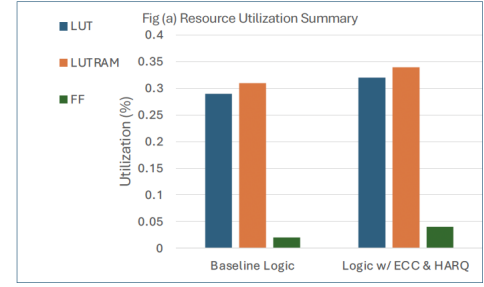


Figure 6. Logic Synthesis Resource Utilization-AXI NOC Baseline Logic w/ECC

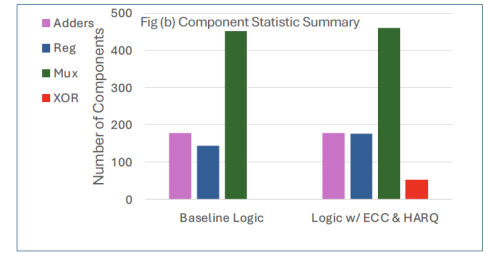


Figure 7. Synthesis, Component Statistics - AXI NOC Baseline Logic w/ ECC

3.1. Static Timing Analysis

Static Timing Analysis (STA) ensures that the timing requirements of the design are met, ensuring correct operation of the circuit under various operating conditions.

3.1.1. Steps in Static Timing Analysis

The STA method begins with a design netlist and time constraints. Critical timing paths are identified, and delays along these paths are estimated using gate delays, wire delays, and flip-flop setup/hold timings. The tool then compares the delays to the limitations and flags any violations.

(b) Hold time violation before area optimization (Baseline)

Setup	Hold
Worst Negative Slack (WNS): 2.365 ns	Worst Hold Slack (WHS): -0.605 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): -803.436 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 6120
Total Number of Endpoints: 9348	Total Number of Endpoints: 10628
Timing constraints are not met.	

Figure 8. STA - Timing report for Baseline Plan 1

Upon observing the initial timing summaries (Eg. for Plan 1 in Figure 8), there were no setup time violations but reports indicated hold time violations. Attempts to fix the violation resulted in modifications to the clock frequency and hold time restrictions which showed no pattern of improvement for the failing endpoints, meaning the hold times were persistent. The conclusion drawn was that there was a possibility of existing false paths which are activated on very rare occasions of inactivity, contributing to the overall critical path. Their delays

can be overlooked since their added paths are considered as exclusions in normal operation. Additionally, the insufficient I/O resources means there are data paths that would be faster than the clock paths as routing to unassigned IOBs are not accounted for, causing hold time violations. Synthesis with custom libraries with no restrictions to available IOB resource would be a promising solution to address this problem.

3.2. Area Optimization

Area optimization is a critical aspect of design efficiency, particularly in scenarios where resource allocation directly impacts overall performance and cost. One approach to achieve this optimization is through the strategic sharing of resources during the conversion process to netlist. By imposing constraints that prioritize component sharing, designers aim to minimize redundant or underutilized elements, thereby reducing the overall area footprint of the design. This strategy entails setting up synthesis directives that prioritize area optimization functions, ensuring that the resulting netlist reflects the desired resource sharing objectives.

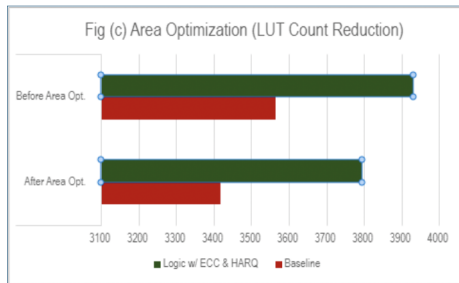


Figure 9. Relate Area optimizations on Vivado Virtex7

The scheme applied for area optimization was the in-built function which can be toggled by setting up the synthesis strategy directive to include the function. After Synthesizing both Baseline Logic before and after area optimization, the LUT count for the Baseline Logic reduced from 3564 to 3417 while the Logic with ECC/HARQ had a reduction in LUT count from 3931 to 3794, indicating an area reduction (by LUT count) of 4.12% in the Baseline design and 3.49% in the Logic with ECC/HARQ.

3.3. Power Optimization

3.3.1. Dynamic Frequency Scaling

Dynamic Frequency Scaling (DFS) in circuits involves dynamically adjusting the operating frequency of processors and components based on workload and performance requirements to optimize power consumption and thermal management. By continuously monitoring workload and performance targets, DFS allows systems to increase frequency during periods of high activity to meet performance demands, while decreasing frequency during idle or low activity periods to conserve power. To mimic dynamic frequency conditions, the operating frequencies of the different interfaces of the design are varied based on Table 10.

3.3.2. Clock gating

Clock gating is a power-saving technique used in circuits to reduce dynamic power consumption by selectively disabling the clock signal to specific circuit elements when they are not

Clock-Freq(MHz)	Mst/Slv0	Mst/Slv1	Mst/Slv2	Mst/Slv3
Clk_plan1	200	100	50	40
Clk_plan2	400	200	100	80
Clk_plan3	600	300	150	120
Clk_plan4	800	400	200	160

Table 1: Different Combination of Clock-ratios representing DFS scenario.

Figure 10. DFS Clock Plan for Constraints

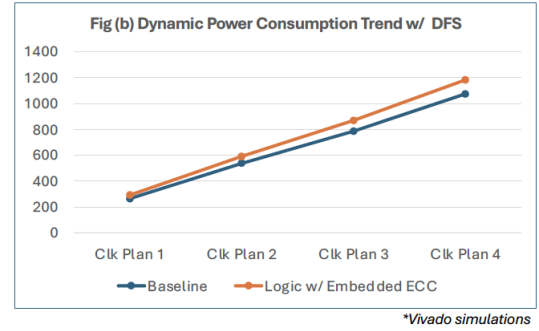


Figure 11. DFS Dynamic Power trend on AXI-NOC

actively processing data. Clock gating is typically implemented using simple logic gates such as AND or NAND gates, which enable or disable the clock signal based on certain conditions or control signals. By effectively shutting off the clock to idle or unused portions of the circuit, clock gating helps to minimize power dissipation and improve overall energy efficiency in VLSI designs without sacrificing performance.

Analysis was made to observe the effect for clock gating on the various clock plans for the baseline logic. Using an extrapolation of 0.5333 mW/MHz/LUT of dynamic power, the dynamic power of both was calculated without considering clock gating. The clock frequencies of the master/slave interfaces were set to 0 MHz by calculation, imitating an inactive and gated Clock 0. Figure 13 show the reduction in consumption as a result of clock gating.

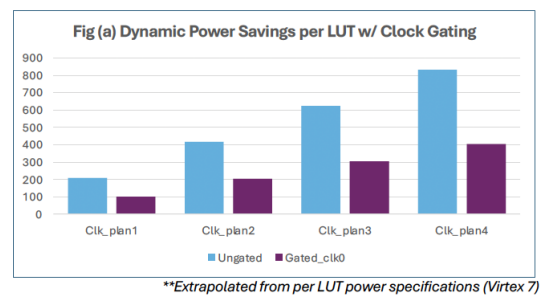


Figure 12. Estimated Dynamic Power Savings per LUT with Clock Gating (to M/S 0 Clock)

4. Test and Verification

Logic verification in VLSI (Very Large-Scale Integration) is the process of ensuring that a digital circuit design behaves as intended and meets its functional requirements. It involves verifying that the logic of the design performs the desired operations correctly under all specified conditions.

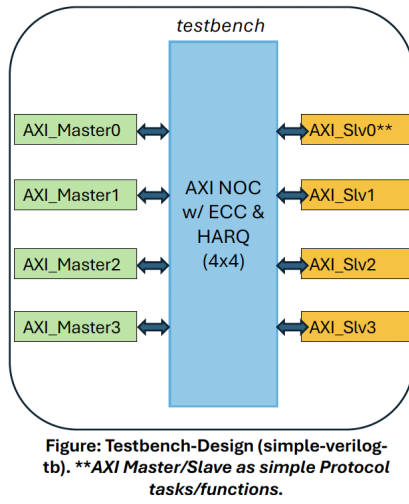


Figure 13. Testbench Design - AXI NOC

4.1. Steps in logic verification

1. **Testbench/Testcase Development:** A testbench is a set of stimuli and test cases designed to exercise the various components of the digital circuit. During logic verification, engineers develop comprehensive testbenches that cover a wide range of input scenarios, including normal operation, edge cases, and corner cases.
2. **Simulation:** The testbenches are used to simulate the behavior of the digital circuit under different input conditions. Simulation tools apply the test stimuli to the circuit model and observe its outputs. Engineers analyze the simulation results to ensure that the circuit behaves correctly and produces the expected outputs for each input scenario.
3. **Functional Coverage Analysis:** Functional coverage analysis evaluates the effectiveness of the testbenches by measuring how well they exercise the various functional aspects of the design. Engineers track the coverage of different logic paths, state transitions, and input combinations to ensure that all critical areas of the design are adequately tested.
4. **Performance Analysis:** Based on the results of performance analysis, designers may implement various optimization techniques to improve the circuit's speed, efficiency, and resource utilization. Optimization techniques may include logic restructuring, pipelining, clock gating, power gating, and technology-specific optimizations.
5. **Power Analysis:** Power verification evaluates the power consumption of the digital circuit under different operating conditions. Engineers use power analysis tools to estimate the dynamic and static power consumption of the design and optimize it to meet power budget constraints.

By performing thorough logic verification, it ensures the correctness, reliability, and performance of digital circuit designs, reducing the risk of errors and improving the quality.

4.2. Verification plans

4.2.1. Functional Verification

Functional verification is essential to ensure the sanity and integrity of the design. In this phase, AXI transactions are

stimulated from master interfaces, and responses are observed from slave interfaces. Additionally, ECC and HARQ verification are conducted by injecting single-bit/double-bit errors using 'force' pragmas in the Verilog testbench. While the focus remains on verifying fundamental functionality, waveforms are utilized during functional verification to validate expected responses from AXI agents. Although complex verification checkers and exhaustive functional coverage are not targeted in this project, this phase ensures the fundamental integrity of the design.

1. **AXI Transactions** - This verification process involves verification the read and write transactions for 16x4 bytes, ensuring the correct handling of random data. By executing read and write operations across the AXI bus, we validate the integrity and reliability of data transfer mechanisms within the system.
2. **Embedded ECC** - In this phase, we assess the error correction capabilities of the system by injecting and detecting single- and double-bit errors. By validating the Single Error Correction Double Error Detection (SECCDED) logic, we ensure that the system can reliably identify and rectify errors in transmitted data.
3. **HARQ Protocol** - Here, we verify the BRESP and RRESP HARQ responses during read and write transactions from the master to the slave. By confirming the proper functioning of the HARQ protocol, we ensure the robustness and efficiency of data communication between master and slave devices.

4.2.2. Performance Verification

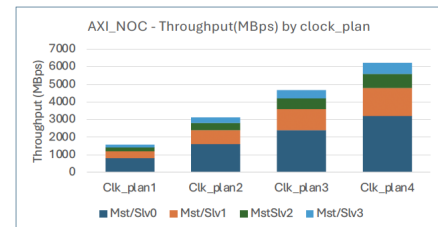
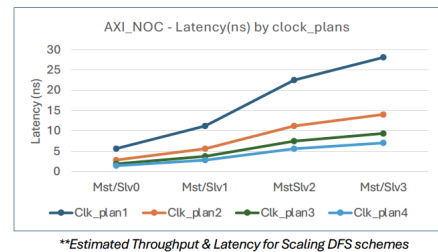


Figure 14. Bandwidth Verification - AXI NOC



**Estimated Throughput & Latency for Scaling DFS schemes

Figure 15. Bandwidth Verification - AXI NOC

1. **Throughput verification** - This verification procedure involves concurrently injecting data from all master interfaces, each operating at different clock rates or combinations thereof. By observing and verifying the throughput of all AXI interfaces, we assess the system's bandwidth capabilities under various workload scenarios.

2. **Latency verification**- In latency verification, data is serially injected from all master interfaces, each utilizing different clock rates or combinations. By observing and verifying point-to-point latency, we evaluate the efficiency and responsiveness of data transmission within the system.

4.2.3. Power Verification

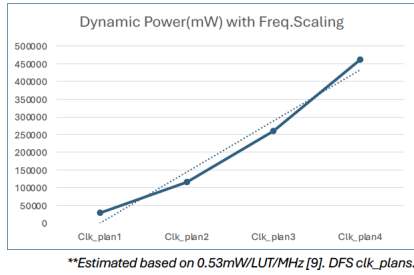


Figure 16. Power Verification, Estimated Dynamic Power with DFS.

1. **Dynamic Frequency Scaling (DFS)** - With Dynamic Frequency Scaling (DFS) utilizing Clock_plan1 frequencies, we concurrently inject data from all master interfaces. By observing and verifying completion on all slave AXI interfaces, we assess the impact of different clock scaling ratios on power consumption and performance.
2. **Clock Gating (clk_gate)** -Clock Gating verification involves gating the clock on master0/slave0 while injecting data from master1/2/3. By verifying transaction completion for Slave1/2/3 under various clock gating combinations, we evaluate the effectiveness of clock gating techniques in reducing power consumption while maintaining system functionality.

5. Future Works

Moving forward, exploring scalability up to 256x256 in high-speed NOCs is crucial, along with investigating various topologies and routing algorithms to ensure adaptability to embedded ECC and HARQ. Additionally, optimizing with custom cell libraries offers potential for performance enhancement beyond FPGA tools like Vivado.

Further validation efforts using methodologies like Coverage-Driven-Random-Verification (CRDV) and assertion-based verification are essential for robustness. Integration of advanced features such as power management schemes and dynamic reconfiguration will enrich the architecture.

6. Conclusion

In conclusion, our study highlights the successful implementation of scalable AXI NOC with embedded ECC/HARQ mechanisms, demonstrating a promising approach to address reliability concerns in contemporary on-chip NOC architectures. By integrating ECC and HARQ logic into a 4x4 NOC configuration, we achieved notable improvements in reliability with minimal resource utilization overhead, as evidenced by the mere 4% increase in FPGA LUT utilization. Additionally, the adoption of multi-stage clock-gating incurred minimal area

overhead while significantly reducing dynamic power consumption by 26% at the 800MHz frequency corner, showcasing the potential for power optimization in high-performance NOC designs.

These findings underscore the practicality and efficacy of our proposed enhancements in enhancing NOC reliability while maintaining resource efficiency. Furthermore, the scalability potential of our approach is evident, as indicated by the suggestion to extend the project to investigate larger-scale NOC configurations, such as 256x256, to accommodate the demands of high-speed communication systems.

Overall, this study contributes to the ongoing research efforts aimed at advancing on-chip communication architectures, providing valuable insights for the design and optimization of reliable and efficient NOC systems in future semiconductor technologies.

7. References

1. S. Kumar et al., "A network on chip architecture and design methodology," Proceedings IEEE Computer Society Annual Symposium on VLSI. New Paradigms for VLSI Systems Design. ISVLSI 2002, Pittsburgh, PA, USA, 2002, pp. 117-124, doi: 10.1109/ISVLSI.2002.1016885
2. Y. Hu, "The Analysis of SECD in Data Storage Transfer and Algorithm Optimization," 2022 International Symposium on Advances in Informatics, Electronics and Education (ISAIEE), Frankfurt, Germany, 2022, pp. 51-54, doi: 10.1109/ISAIEE57420.2022.00018
3. Mengjia Yan, On-Chip Networks I: Topology/Flow Control, MIT 6.823 Spring 2020 Lecture
4. Jiang Xu, W. Wolf, J. Henkel, and S. Chakradhar, 'A Methodology for Design, Modeling, and Analysis of Networks-on-Chip', in 2005 IEEE International Symposium on Circuits and Systems, Kobe, Japan: IEEE, 2005, pp. 1778-1781. doi: 10.1109/ISCAS.2005.1464953.
5. G. Reehal and M. Ismail, 'A Systematic Design Methodology for Low-Power NoCs', IEEE Trans. VLSI Syst., vol. 22, no. 12, pp. 2585-2595, Dec. 2014, doi: 10.1109/TVLSI.2013.2296742
6. Onur Multu, Computer Architecture, ETH Zürich - Lecture 26: On-Chip Networks (Fall 2023).
7. J. H. Anderson and F. N. Najm, "Power-aware technology mapping for LUT-based FPGAs," 2002 IEEE International Conference on Field-Programmable Technology, 2002. (FPT). Proceedings., Hong Kong, China, 2002, pp. 211-218, doi: 10.1109/FPT.2002.1188684.
8. Xilinx Vitrix 7 -II datasheet: https://docs.amd.com/v/u/en-US/ds183_Virtex_7_Data_Sheet